# Algorithms for NP-Hard Problems

Summary note, Lectured by Eunjung Kim, KAIST

**Hojune Lee**

**School of Computing, KAIST**

# Contents

**1 Parameterized Algorithm** **3**

**2 Branching Algorithms** **5**

2.1 $\mathcal{O}(2^k)$-time Algorithm for Vertex Cover . . . . . 5

2.2 $\mathcal{O}((3k)^k)$-time Algorithm for Feedback Vertex Set . . . . . 6

2.3 $\mathcal{O}(4^{k-lp^*(G)})$-time Algorithm for Vertex Cover . . . . . 8

# Chapter 1

# Parameterized Algorithm

**Parameterization**

The *parameterization* is a map defined by

$$\kappa : \Sigma^* \to \mathbb{N}_0$$

With this definition, *parameterized problem* is an ordered pair decision problem for language $\mathfrak{L} \subseteq \Sigma^*$ and parameterization $\kappa$, $(\mathfrak{L}, \kappa)$. Since $\kappa$ is a map, we can take any well-defined function for $\kappa$. So, we can claim that for wrong encoded instance in $\Sigma^*$, they goes to some fixed value, e.g. 0.
Here, it is important that why we take this definition. I'll write the intuition later, after introducing the example.

**Vertex Cover**

All problems are defined in red-colored box. First problem is Vertex Cover. For a graph $G = (V, E)$ and $k \in \mathbb{N}_0$, *vertex cover* is a set $S \subseteq V$ such that $\forall uv \in E, u \in S \vee v \in S$. i.e. All edges in $E$ are covered by $S$.
Here, Vertex Cover is a decision problem of following encoded language $\mathfrak{L}$ :

$$\mathfrak{L}_{VC} = \{(G, k) | G = (V, E), k \in \mathbb{N}_0, G \text{ has vertex cover whose size at most } k\}$$

Since $(G, k) \in \Sigma^*$ is a encoded string, they are together. Means that, when we describe the input size $|x|$ of algorithm, it doesn't contain the seperated meaning of size of $G$ and magnitude of $k$. So we parameterization $\kappa$ here, to extract the meaning of $k$.

$$\kappa : (G, k) \mapsto k$$

As we'll see later, another information also can be extracted :

$$\kappa : (G, k) \mapsto k - lp^*(G)$$

This is conceptual meaning of parameterization of a problem.

**Fixed Parameter Tractable**

A parameterized problem $(\mathfrak{L}, \kappa)$ is *fixed parameter tractable* if there is an algorithm such that for every $x \in \Sigma^*$, decides whether $x \in \mathfrak{L}$ or not in time $f(\kappa(|x|)) \cdot |x|^{\mathcal{O}(1)}$

# Chapter 2

# Branching Algorithms

## 2.1 $\mathcal{O}(2^k)$-time Algorithm for Vertex Cover

**Algorithm 1** $\mathcal{O}(2^k)$-time Algorithm for Vertex Cover

1: **procedure VC**$(G, k)$

**Input:** $G = (V, E)$ is a graph, $k \in \mathbb{N}_0$

**Output:** Decide $(G, k) \in \mathfrak{L}_{VC}$ or not

2: **if** $|E| = 0$ **then return** Yes

3: **else if** $k = 0$ **then return** No

4: **else**

5: $uv \leftarrow Pick(E)$ ▷ Pick any edge

6: **return VC**$(G - u, k - 1) \vee$ **VC**$(G - v, k - 1)$

This is very fundamental algorithm for Vertex Cover. We can prove following:

**Algorithm 1**

Algorithm 1 is *correct*(i.e., decides whether $(G, k) \in \mathfrak{L}_{VC}$ or not correctly) in time $\mathcal{O}(2^k \cdot poly(n))$

*Proof.* We can say that algorithm 1 is correct means that

$$\mathbf{VC}(G, k) = \text{Yes} \iff (G, k) \in \mathfrak{L}_{VC}$$

We'll prove via mathematical induction on $k \in \mathbb{N}_0$. If $k = 0$, it is trivial. Then assume that for any $G = (V, E)$, $VC(G, i) = \text{Yes} \iff (G, i) \in \mathfrak{L}_{VC}, 0 \leq i < k$. And one easy fact is that, if $S$ is a vertex cover of $G$, and $u \in S$, then $S \setminus \{u\}$ is a vertex cover of $G - u$.
($\Leftarrow$) Let $(G, k) \in fL_{VC}$. Then, exists a vertex cover $S$ of $G$ such that $|S| \leq k$. Suppose that given instance is non-trivial, so reach at line 4. Then pick $\forall uv \in E$, and W.L.O.G., $u \in S$. Then by fact, $S \setminus \{u\}$ is a vertex cover of $G - u$ whose size is at most $k - 1$. Then, we can determine that $\mathbf{VC}(G - u, k - 1) = \text{Yes}$ by hypothesis.
($\Rightarrow$) Suppose that $\mathbf{VC}(G, k) = \text{Yes}$. Then for non-trivial cases, W.L.O.G., $\mathbf{VC}(G - u, k -$

1) is Yes. It implies that $(G-u, k-1) \in \mathfrak{L}_{VC}$. Then, let $S'$ be the vertex cover of $G-u$ whose size is at most $k-1$. Then, let $S = S' \cup \{u\}$. It is clearly a vertex cover of $G$ whose size is at most $k$. So $(G, k) \in \mathfrak{L}_{VC}$.

Now, for the running time. It is obvious that line 2,3,5 effects polynomial time w.r.t. input size. And there are at most $k$-depth of binary recursion, so running time is $\mathcal{O}(2^k \cdot poly(n))$.

□

## 2.2 $\mathcal{O}((3k)^k)$-time Algorithm for Feedback Vertex Set

Now we define next problem to solve, which is called Feedback Vertex Set.

**Feedback Vertex Set**

For any graph $G = (V, E)$, a *feed-back vertex set* of $G$ is a set $S \subseteq V$ such that

$$\forall C \in Cycle(G), S \cap C \neq \emptyset$$

i.e., set of vertices that intersect with all of cycles in $G$. So, we can define decision problem Feedback Vertex Set by

$$\mathfrak{L}_{FVS} = \{(G, k) | G = (V, E), k \in \mathbb{N}_0, G \text{ has FVS whose size at most } k\}$$

**Algorithm 2** $\mathcal{O}((3k)^k)$-time Algorithm for Feedback Vertex Set

```
1: procedure FVS(G, k)
2:     if |E| = 0 then return Yes
3:     if k = 0 then return No
4:     while ∃v ∈ V(G), d_G(v) ≤ 2 do
5:         if d_G(v) ≤ 1 then
6:             G ← G − v
7:         else if |N_G(v) \ {v}| = 2 then
8:             {x, y} ← N_G(v) \ {v}
9:             G ← G − v + xy                          ▷ Allow multigraphs
10:        else
11:            G ← G \ N_G(v)
12:            k ← k − 1          ▷ Assume that we choose the adjacent vertex of v
13:    {v_n, ··· , v_1} ← Sort(V(G))                   ▷ Note: Decreasing Order
14:    return ⋁_{i=1}^{3k} FVS(G − v_i, k − 1)
```

**Algorithm 2**

Algorithm 2 is correct for FEEDBACK VERTEX SET in time $\mathcal{O}((3k)^k \cdot poly(n))$

**Lemma 2.2.1.** *For any feed-back vertex set $X$ of $G = (V, E)$ holds that*

$$\sum_{x \in X} (d_G(x) - 1) \geq |E| - |V| + 1$$

*Proof.* One easily obtained fact is that, $G[V \setminus X]$ is a forest. Means that,

$$|E(G[V \setminus X])| \leq |V(G[V \setminus X])| - 1 = |V| - |X| - 1$$

However,

$$|E| = \underbrace{|E(G[X])| + |E(X, V \setminus X)|}_{\leq \sum_{x \in X} d_G(x)} + |E(G[V \setminus X])|$$

So,

$$\sum_{x \in X} d_G(x) \geq |E| - |E(G[V \setminus X])| \geq |E| - |V| + |X| + 1$$

Implies that

$$\sum_{x \in X} (d_G(x) - 1) \geq |E| - |V| + 1$$

□

**Lemma 2.2.2.** *Let $G = (V, E)$ such that $\delta(G) \geq 3$. Let $V = \{v_1, v_2, \cdots, v_n\}$ where*

$$d_G(v_1) \geq d_G(v_2) \geq \cdots \geq d_G(v_n)$$

*Then, for any feedback vertex set $X$ of $G$,*

$$|X| \leq k \implies X \cap \{v_1, \cdots, v_{3k}\} \neq \emptyset$$

*Proof.* Suppose not. By hand-shake lemma,

$$\sum_{v \in V} d_G(v) = 2|E| \geq 3|V| \qquad \because \delta(G) \geq 3$$

And since $X \cap \{v_1, \cdots, v_{3k}\} = \emptyset$, We can obtain following from above lemma:

$$\sum_{i=1}^{3k} (d_G(v_i) - 1) \geq 3 \sum_{x \in X} (d_G(x) - 1) \geq 3|E| - 3|V| + 3$$

$$\sum_{i=3k+1}^{n} (d_G(v_i) - 1) \geq \sum_{x \in X} (d_G(x) - 1) \geq |E| - |V| + 1$$

implies that,

$$\sum_{v \in V(G)} (d_G(v) - 1) = 2|E| - |V| \geq 4|E| - 4|V| + 4$$

It deduce

$$3|V| > 2|E|$$

which contradict to first line. □

With second lemma, one can show that Algorithm 2 is correct. And since it generates at most $3k$ branches for each step, and the maximal depth is $k$, so the running time will be $\mathcal{O}((3k)^k \cdot poly(n))$

## 2.3 $\mathcal{O}(4^{k-lp^*(G)})$-time Algorithm for Vertex Cover

In this section, we review the VERTEX COVER with another parameterization, which is

$$\kappa : (G, k) \mapsto k - lp^*(G)$$

Here, $lp^*(G)$ is a solution of another problem, which called linear program version of vertex cover.

**Linear Program for Vertex Cover**

For $G = (V, E)$, linear program for vertex cover is called LPVC, which defined as following.

$$\min \sum_{u \in V(G)} x_u$$

where

$$\forall uv \in E, x_u + x_v \geq 1, \qquad \forall u \in V, x_u \geq 0 \tag{$\star$}$$

holds. Let's call above condition by $\star$. And we denote $lp^*(G)$ be the optimal solution of LPVC.

One fact is that,

$$\text{size of optimal vertex cover of } G \geq lp^*(G)$$

This is because that, optimal vertex cover is very special case of LPVC. It can be easily obtain, since every vertex cover satisfy the condition $\star$. So, this automatically implies that for given $(G, k)$, if $k < lp^*(G)$, $(G, k)$ is No instance of VERTEX COVER.

Now, we'll explore the algorithm which related to these LP problems. Before move on, we'll take a following theorem without proof.

**Linear Program is P**

Linear Program can be solved in polynomial time. So that, LPVC also solved in polynomial time.

And now we'll define half-integral solution for LPVC.

**Half Integral solution for LPVC**

We call that the solution $x^*$ for LPVC is *half-integral* when

$$\forall u \in V(G), x_u^* \in \{0, \frac{1}{2}, 1\}$$

With the above fact, we can conduct following decomposition in polynomial time.

**Crown Decomposition**

Let $x^*$ be the solution of LPVC. Then we can partition $V(G)$ into following 3 sets :

$$C = \{u \in V(G) | x_u^* < 0.5\}$$

$$H = \{u \in V(G) | x_u^* > 0.5\}$$

$$R = \{u \in V(G) | x_u^* = 0.5\}$$

This is called *crown decomposition* of $G$. This decomposition can be obtained in polynomial time.

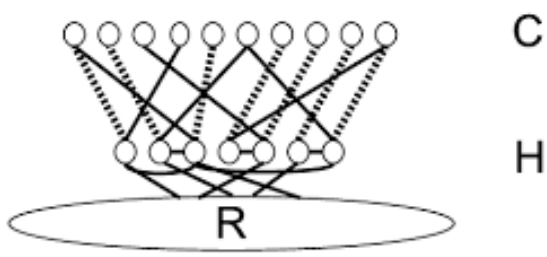


**Figure 2.1:** A graph illustrating the crown decomposition.

Now, we are going to argue many properties on crown decomposition.

**Lemma 2.3.1.** *For every $H' \subseteq H$, $|N_G(H') \cap C| \geq |H'|$*

*Proof.* Actually, this is quite intuitive. If not, that situation is very un-natural to reduce the total-sum.

Suppose not, i.e., there exists $H' \subseteq H$ such that $|N_G(H') \cap C| < |H'|$. Now idea is very simple, let's construct more cheap solution from here! We can define following value $\epsilon > 0$

$$\epsilon = \min_{h \in H'} \{x_h^* - 0.5\} > 0$$

And then, we'll construct more cheap solution by

$$\forall u \in V(G), x'_u = \begin{cases} x_u^* + \epsilon & u \in N_G(H') \cap C \\ x_u^* - \epsilon & u \in H' \\ x_u^* & \text{otherwise} \end{cases}$$

$x'$ is feasible LP solution since all our reponse to check is for incident parts of $H'$, but it preserves $H'$ to $C$ sum, and since $x'$ have same crown decomposition, $H'$ to $H$ and $H'$ to $R$ also resolved. However, as $|N_G(H') \cap C| < |H'|$, the total cost $|x'|$ is strictly small than $|x^*|$.

$$\sum_{u \in V(G)} x'_u = \left( \sum_{u \in V(G)} x_u^* \right) - \epsilon(|H'| - |N_G(H') \cap C|) < \sum_{u \in V(G)} x_u^*$$

This contradicts to $x^*$ is an optimal solution.

□

**Lemma 2.3.2.** *Let $(R, H, C)$ be a crown decomposition of $G$ w.r.t. optimal solution $x^*$ of* LPVC. *Then, following solution is also a optimal solution of* LPVC.

$$x'_u = \begin{cases} 0.5 & u \in R \\ 1 & u \in H \\ 0 & u \in C \end{cases}$$

*Proof.* Let $\mathcal{X}$ be the set of all optimal solution of LPVC such that yields crown decomposition $(R, H, C)$. And for each $x^* \in \mathcal{X}$, we can define following two sets:

$$C_{x^*} = \{c | c \in C, x^*_c > 0\}$$

$$H_{x^*} = \{h | h \in H, x^*_h < 1\}$$

Then we can change our claim into that there exists $x' \in \mathcal{X}$ such that $C_{x'} \cap H_{x'} = \emptyset$. Suppose not, and let $x'$ be the solution that minimize $|C_{x'} \cap H_{x'}| > 0$. In this case, we can choose following $\delta > 0$.

$$\delta = \min\left(\min_{c \in C_{x'}}\{x'_c\}, \min_{h \in H_{x'}}\{1 - x'_h\}\right)$$

Then here, we can construct following revised solution $x''$ by

$$x''_u = \begin{cases} x'_u - \delta & u \in H_{x'} \\ x'_u + \delta & u \in C_{x'} \\ x'_u & \text{otherwise} \end{cases}$$

It is obvious that $x''$ is feasible solution of LPVC. Actually, $x''$ is an optimal solution. First, for all vertices in $H_{x'}$ can't be incident with $C \setminus C_{x'}$. (Because, vertices in $C_{x'}$ assigned 0) It implies that, $N_G(H_{x'}) \cap C \subseteq C_{x'}$. However, we know that $|H_{x'}| \leq |N_G(H_{x'}) \cap C|$ by above lemma. So finally, we these two relation,

$$|H_{x'}| \leq |C_{x'}| \implies |x''| \leq |x'|$$

Since $x'$ is optimal, $|x''| = |x'|$ and so $x''$ also optimal solution. However, at least one vertex in $H_{x'}$ or $C_{x'}$ are collapsed into 1, 0 respectively, so $|C_{x''} \cap H_{x''}| < |C_{x'} \cap H_{x'}$ which contradicts to choice of $x'$. So that, there exists $x' \in \mathcal{X}$ such that $C_{x'} \cap H_{x'} = \emptyset$, holds our claim. □

**Remark** During the proof, we construct the algorithmic way to convert any optimal LPVC solutions into half-integral optimal LPVC solution. Since we can obtain any LPVC solution in polynomial time and above algorithm also, we can find half-integral optimal LPVC solution $x^*$ in polynomial time always.

**Obtain Half-Integral Solution**

For given graph $G = (V, E)$, we can obtain half-integral optimal LPVC solution $x^*$ in polynomial time. And define

$$V_a(x^*) = \{v \in V | x_v^* = a\}$$

Then here, crown decomposition w.r.t. $x^*$ is $(V_{0.5}, V_1, V_0)$.

**Lemma 2.3.3.** *For $G' = G[V_{0.5}]$, $\forall v \in G', x'_v = 0.5$ is an optimal solution of LPVC of $G'$.*

*Proof.* Very easy. We can observe that, on the crown decomposition of $G$, all $V_{0.5}$ vertices are incident with $V_{0.5}$ itself or $V_1$. So, if there exists another optimal solution than $x'$, we can transplant that solution directly in $G$, then we can make more optimal solution to LPVC of $G$. This occur contradiction, so $x'$ is an optimal solution of LPVC of $G[V_{0.5}]$. □

**Lemma 2.3.4.** *There exists an optimal vertex cover containing $V_1$.*

*Proof.* Let $S$ be any optimal vertex cover of $G$. Then, w.r.t. the crown decomposition of $G$ by $(V_{0.5}, V_1, V_0)$, we can define

$$S_0 = S \cap V_0, \quad S_{0.5} = S \cap V_{0.5}, \quad S_1 = S \cap V_1$$

Then let's construct new set

$$S' = (S \setminus S_0) \cup V_1$$

i.e., replace all cover vertices in $C$ into remaining vertices in $H$. One can be easily observed, $S'$ is a vertex cover of $G$. So, to check $S'$ also an optimal vertex cover, we just need to check that number of removed vertices ($|S_0|$) is larger or equal than number of added vertices ($|V_1 \setminus S_1|$). Not too hard to see. Imagine the crown decomposition diagram. Since $S$ was vertex cover, all the edges are covered. It means that, there aren't any edges between $V_0 \setminus S_0$ and $V_1 \setminus S_1$. So, $N(V_1 \setminus S_1) \cap V_0 \subseteq S_0$. And according to the Lemma 2.3.1.,

$$|V_1 \setminus S_1| \leq |N(V_1 \setminus S_1) \cap V_0| \leq |S_0|$$

This completes the proof. □

**Lemma 2.3.5.** *The instance $I' = (G', k')$ is equivalent to $I = (G, k)$ where $G' = G[V_{0.5}]$ and $k' = k - |V_1|$. Furthermore, it holds that $k' - lp^*(G[V_{0.5}]) = k - lp^*(G)$*

*Proof.* Let's prove that $I' \iff I$ first.

($\Rightarrow$) Suppose that $(G', k')$ is Yes instance. Then, take a vertex cover $S'$ for $G'$ whose size is at most $k'$. Then, for $G$, just set $S = S' \cup V_1$. It is clearly vertex cover of $G$ and

$$|S| = |S'| + |V_1| \leq k' + |V_1| = k$$

so $(G, k)$ is Yes instance.

($\Leftarrow$) Suppose that $(G, k)$ is Yes instance. Then, by Leamma 2.3.4. there exists an optimal vertex cover $S$ containing $V_1$. Then, $S_{1/2} = S \setminus V_1 \subseteq V_{1/2}$ holds. Then, this cover all the vertices in $V_{1/2} \setminus S_{1/2}$ since if not, it is contradicts to optimal LPVC solution. So, $S_{1/2}$ is vertex cover of $G[V_{1/2}]$. However, since $|S| \leq k$,

$$|S_{1/2}| = |S| - |V_1| \leq k - |V_1| = k'$$

So, $(G', k')$ is Yes instance.

For the further statement, we already have lemma that $lp^*(G[V_{0.5}]) = \frac{1}{2}|V_{1/2}|$. And for optimal LPVC half-integral solution, $lp^*(G) = |V_1| + \frac{1}{2}|V_{1/2}|$. So the claim holds. $\square$

**Remark** We show that $\forall v \in V_{1/2}, x_v^* = 1/2$ can be optimal LPVC solution for $G[V_{1/2}]$. It does not guarantee that it is unique.

Now, with Lemma 2.3.5. we can reduce(or preprocess) our original instance $(G, k)$.

**Reduction Rule of LPVC**

Let $(G, k)$ be given instance.

1. For every half-integral optimal LPVC solution $y^*$ of $(G, k)$,
2. If there aren't any vertex $v$ such that $y_v^* = 1$, then end.
3. If such $v$ exists, construct $(G[V_{1/2}(y^*)], k - |V_1(y^*)|)$

It holds because of Lemma 2.3.5.

With apply forever this rule, we finally obtain $(G', k')$ whose optimal half-integral LPVC solution is $\forall v \in V(G'), x_v^* = 1/2$. Then, we can construct new Vertex Cover algorithm.

**Algorithm 3** $\mathcal{O}(4^{k-lp^*(G)})$-time algorithm for Vertex Cover

```
procedure VC-above-LP(G, k)
    (G, k) ← Apply reduction rule until converge
    // Here, all-1/2 is the unique half-integral optimal solution.
    if k < lp*(G) = 1/2|V(G)| then
        return No                                   ▷ We observed this fact
    else if k ≥ 2 · lp*(G) = |V(G)| then
        return Yes                                  ▷ Because we can pick every vertices
    else
        uv ← Pick(E(G))
        return VC-above-LP(G − u, k − 1) ∨ VC-above-LP(G − v, k − 1)
```

It is easy to see that, this algorithm works correctly. Let's consider following measure:

$$\mu(G, k) = k - lp^*(G)$$

i.e., this is our new parameterization for Vertex Cover above LP. Then, our goal is that, by showing this factor decreases at least $\frac{1}{2}$ per each step, so our algorithm terminated and depth of tree is at most $2(k - lp^*(G))$.

**Proposition 2.3.6.** *For every iteration, the measure $\mu$ decreases at least $\frac{1}{2}$.*

*Proof.* Here we claim that, for every $v \in V(G)$,

$$lp^*(G - v) \geq lp^*(G) - \frac{1}{2}$$

Suppose not. Then, $lp^*(G-v) < lp^*(G) - \frac{1}{2}$. Since each $lp^*$ value is quantumized by 1/2, it implies that

$$lp^*(G - v) \leq lp^*(G) - 1$$

Then, let's choose the half-integral optimal LPVC solution $x^*$ of $G - v$. Then, append $x^*$ for $v$ with $x^*_v = 1$. Then such new $x^*$ holds size $lp^*(G)$ optimal solution of LPVC of $G$, however, all-1/2 is unique solution. It is contradiction, so claim holds. With that, for every iteration, W.L.O.G. for vertex $v$,

$$\mu(G - v, k - 1) = k - 1 - lp^*(G - v) \leq k - lp^*(G) - \frac{1}{2} = \mu(G, k) - \frac{1}{2}$$

However, our algorithm guarantees that if $k < lp^*(G)$ or $k \geq 2 \cdot lp^*(G)$ then terminate. So, if $k - lp^*(G) < 0$, then terminate. So we can decrease our measure at most $2\mu(G, k)$ times. So the running time is $\mathcal{O}(4^{k-lp^*(G)})$. And also, we guaranteed that $k - lp^*(G) \leq k/2$. So, it works as well as $\mathcal{O}(2^k)$ time.

□